# Evaluación 1 - Taller de Programación
# Problema del juego COLOR BLOCK JAM generalizado

Prof. Pablo Román

## 1 Objetivo

Desarrollar una aplicación eficiente y heurística para resolver el problema del juego Color Block Jam, basada en el algoritmo A*. Se implementará en el lenguaje C++ (sin utilizar contenedores o algoritmos de STL), el uso de makefile, y la estructuración, buenas prácticas de un código orientado a objetos.

## 2 Algoritmo A*

Este algoritmo se verá en detalle en clases. Este tipo de procedimientos se utilizan cuando la respuesta a un problema determinado corresponde a una secuencia de pasos a realizar (`https://en.wikipedia.org/wiki/A*_search_algorithm`). El algoritmo se basa en la noción de estado; este contiene el valor de cada variable del problema. Entonces, los pasos corresponden a cambios en los valores de las variables de dicho estado. Esto conforma un grafo, donde cada nodo es un estado con valores particulares de dichas variables y los pasos son conexiones dirigidas entre nodos. Adicionalmente, existe un estado de inicio del sistema y una condición para el estado final. La solución corresponde a la descripción de un camino entre el estado inicial y alguna solución final.

Notar que el diseño del nodo, de la estructura de datos que lo implementa, va a ser parte fundamental del algoritmo. El estado debe representar el sistema de manera completa y unívoca. El sistema cambia o muta cuando se realizan ciertas operaciones sobre el sistema que cambian su representación como estado. La complejidad de dichas operaciones en cada estado dependerá del diseño de este. El algoritmo A* genera todas las combinaciones de operaciones sobre el estado de inicio que puedan ser mantenidas en memoria. Por esta razón, en el peor caso es un algoritmo de fuerza bruta. Sin embargo, si las combinaciones se generan con un criterio de llegar a la solución final lo más rápido posible, entonces se evitarán combinaciones que sean menos conducentes.

El algoritmo A* (alg 2) va construyendo el grafo a medida que se va ejecutando y se asegura de no repetir nodos ya visitados. Para ello, mantiene una estructura de datos para los nodos por visitar y los ya visitados.

**Algorithm 1** A* Search

```
Require: (start, goal)
  OpenSet = {start}
  ClosedSet = ∅
  while OpenSet is not empty do
      current = OpenSet→ pop()
      ClosedSet→ push(current)
      if current = goal then
          return reconstruct path from goal to start
      end if
      for each neighbor of current do
          if neighbor ∈ ClosedSet OR neighbor is not valid then
              continue
          end if
          OpenSet→ push(neighbor)
      end for
  end while
  return not Found
```

El algoritmo A* permite resolver combinaciones de operaciones para llegar a cierto resultado. En la práctica, dicho algoritmo se utiliza cuando se dispone de un conjunto de operaciones que modifican el estado del sistema. En ingeniería existe una infinidad de aplicaciones en las cuales se utiliza. Por ejemplo, el caso de un robot autónomo que requiere llegar a cierto destino dado un mapa donde las operaciones son pasos en ciertas direcciones. En seguridad informática, cuando se debe chequear si se pueden sortear las barreras de seguridad, se busca una secuencia de operaciones que logre vulnerar el sistema. En gestión de operaciones, se busca encontrar la secuencia de operaciones de compra/venta que permita lograr cierto objetivo. Otra aplicación clásica es resolver el cubo Rubik o el puzzle 8.

# 3 Problema a resolver: Juego Colored Block Jam (CBJ)

Esta tarea consiste en resolver el juego de Colored Block Jam (CBJ `https://play.google.com/store/apps/details?id=com.GybeGames.ColorBlockJam&hl=en`) de manera obligatoria utilizando el algoritmo A*, junto con heurísticas para acelerar la búsqueda de una solución. El sistema es un arreglo rectangular que contiene bloques coloreados, paredes, salidas coloreadas y de tamaño variable en los bordes del tablero. La idea es mover los bloques por el tablero cumpliendo las reglas del juego, por ejemplo: no superponer bloques con otros objetos. Cada bloque ocupa varias celdas y puede moverse según las reglas del juego (`https://youtu.be/JkGv4fEBlzE?si=JkjRPmP-ZaZHx1YJ`). Esta tarea no refleja completamente este juego; está adaptada para fines de esta evaluación.

De manera formal, el tablero es un arreglo de $H \times W$. Cada celda $(i, j)$ del tablero contiene de manera somera los siguientes contenidos:

1. valor vacío: Es el fondo del tablero por el cual se pueden mover los bloques.
2. muralla: Es un lugar donde ningún bloque se puede mover.
3. salida para cierto color: Es un lugar donde podría un bloque poder salir.
4. compuerta para cierto color: Es un lugar que puede atravesar un bloque del color correspondiente.
5. bloque: Es parte de una pieza móvil llamada bloque.

La siguiente figura muestra un ejemplo de dicho juego.

Figure 1: Tablero de CBJ (Fuente: `https://play.google.com/store/apps/details?id=com.GybeGames.ColorBlockJam`)

El tablero cambia el contenido de sus celdas en la medida en que el juego va avanzando. Se dispone de un tiempo que se contabiliza según la cantidad de celdas en las cuales se desliza el bloque que se está jugando. El jugador desliza los bloques a voluntad causando que el tiempo avance y realizando los siguientes cambios en el tablero:

- Los bloques al deslizarse, dejan celdas vacías donde estaban y llenan las celdas vacías hacia donde se mueven.
- Los bloques solo pueden deslizarse horizontalmente y verticalmente en ambos sentidos. No pueden pasar por encima de otros bloques o paredes. En un movimiento pueden avanzar una o varias celdas si es que hay espacio disponible.
- Un bloque sale del tablero cuando queda adyacente (no hay celdas entre ambos) a una salida que corresponde a su color y además tiene el mismo ancho o alto que esta. Si una de esas dos condiciones no se cumple, entonces no sale. Al salir un bloque, todas las celdas que ocupaba quedan vacías.

- Las compuertas van cambiando de color en cada paso de tiempo y en un orden predefinido. Cuando un bloque está adyacente a una compuerta, sus colores coinciden y sus anchos o altos coinciden (es decir, cabe), entonces pasa al otro lado solo si hay suficiente espacio para que pase completamente. Es decir, en un paso pasa por completo al otro lado.
- Hay salidas que cambian de tamaño con el tiempo. Para ello cambian a pared cuando se achican o al color respectivo cuando se agrandan. El cambio de tamaño se realiza por paso de tiempo en diferentes configuraciones.
- Hay bloques que no se conoce su color hasta que haya salido una cierta cantidad de bloques. Cada vez que sale un bloque se descuenta una unidad de los bloques que quedan. Cuando un bloque de esta característica su contador llega a 0, entonces se revela su color y puede pasar por compuertas o salidas respectivas.

Cada juego tiene un tiempo máximo disponible, contabilizado en cantidad total de celdas para deslizar bloques. Los bloques tienen distintas propiedades:

- ID: Es un número que permite identificar al bloque que se mueve.
- Ancho $w$ y Alto $h$: Esto genera un subarreglo de $w \times h$.
- Color del bloque: Es un número natural. Para fines de visualización puede ser un carácter de letra minúscula trasladado a codificación ascii.
- Bloqueo de color: 0 indica no bloqueado y número entre 1 y 9 indica bloqueado por cierta cantidad de salidas.
- Geometría: indica la forma explícita del bloque. Para ello se indica un arreglo booleano de $w \times h$, donde 1 indica celda parte del bloque y 0 celda externa al bloque. Notar que ninguna celda en 1 puede estar encima de una celda ocupada por otro bloque, compuerta o pared; en caso contrario son los 0.
- Posición $(x, y)$ en el tablero: Son la fila y columna donde se ubica la esquina $(0, 0)$ del bloque en el tablero.

Las compuertas tienen distintas propiedades:

- Son parte de una pared, por lo que pueden ser vistas como una pared con propiedad de compuerta. Es decir, con una bandera booleana llamada compuerta.
- Color inicial $c_i$: color con el que se inicia en el juego.
- Color de fin de ciclo $c_f$: Color con el que termina de ciclar y después vuelve a $c_i$. Por ejemplo si $c_i == c_f$ entonces se queda en el mismo color.
- Número de pasos $p$ en que cambia el color: cada $p$ pasos de tiempo cambia el color, por lo que se suma 1 al color actual. Por ejemplo, si $p == 0$ el color no cambia. Si el color actual es $c_f$ el próximo color será $c_i$.
- De lo anterior se deduce que no pueden estar en un borde del tablero, porque no llevaría a ningún lado.

Las salidas del tablero tienen las siguientes propiedades:

- Se encuentran ubicadas en las paredes del tablero, y tienen una ubicación $(x, y)$ definida. Eso significa que una pared puede tener propiedades de salida. Eso se podría controlar con una variable boolean que indique si cierta celda es salida o no. Pero una pared nunca puede ser simultaneamente salida y compuerta.
- Una salida tiene un color asociado y es constante en el tiempo.
- Una salida tiene un largo $L$: Cuando un bloque sale por el lado superior/inferior o izquierdo/derecho debe tener un ancho o alto respectivamente igual o menor que el largo de la salida. Este largo involucra varias celdas.
- El largo puede fluctuar con el tiempo: Eso significa que parte de la secuencia de celdas de largo L se van a comportar como pared (pierden color). La convención es que las celdas se apaguen desde aquellas con mayor coordenada y se repite incrementando en forma periódica. Para lo cual se parametriza como: $L_i$ largo inicial, $L_f largofinal$, $p$ número de pasos en que cambia el largo. Por ejemplo si $L_i == L_f == L$ entonces el tamaño no cambia. Si por ejemplo $L_i = 8$, $Lf = 5$, $p = 1$, entonces la secuencia de largos será $8, 7, 6, 5, 6, 7, 8, 7 \ldots$

El juego termina con una solución si todos los bloques lograron salir del tablero. Si no es posible se retorna que no hay solución. La última y no menor restricción es que hay un número de pasos límite $NP$. Desde el punto de vista computacional, es un alivio ya que alivia un poco la explosión combinatorial del problema.

# 4 Implementación

Se exige que la implementación sea eficiente y escrita de manera limpia y clara, en el lenguaje de programación C++. Para esta primera tarea **no se podrán utilizar los contenedores y algoritmos de la librería STL**. Se espera que implementen todas las estructuras de datos en una clase por cada una de ellas. Se debe cuidar la orientación al objeto: No deben haber funciones sueltas, solo clases y funciones main para test y programa principal. Es importante que el programa final resuelva en forma eficiente el problema. Cada clase debe tener asociado un programa para fines de test.

El programa principal (main.cpp) deberá desplegar un menú, donde una de sus opciones es "Salir". Este menú también incluye una opción para cargar un archivo de configuración que contiene la información necesaria que describe el tablero de CBJ, su disposición inicial y valores necesarios para este juego. La salida que se debe mostrar después de seleccionar en el menú la opción que resuelve el problema es un listado de operaciones U:Up D:Down R:Right L:Left que indican los movimientos de un bloque junto al ID del bloque a mover y cantidad de celdas a mover. Ejemplo: U1,2D2,1R1,10 indica que bloque 1 se mueve 2 celdas hacia arriba, bloque 2 se mueve una celda hacia abajo, bloque 1 se mueve 10 celdas a la derecha. Se indica el tiempo en milisegundos que se utilizó para resolver, esto es en el caso de que el tablero tenga solución. En el caso de que no tenga solución se muestra un mensaje "Juego sin solución" y el tiempo asociado. El menú debe tener además, opciones para: mostrar el tablero según los pasos en que el juego se resuelve, mostrar los pasos en que se modifica el tablero desde el inicio si se ingresa una secuencia de movimientos como el ejemplo anterior.

El archivo de configuración tiene la siguiente sintaxis:

```
[META]
NAME = <string>
WIDTH = <int>
HEIGHT = <int>
STEP_LIMIT = <int>

[BLOCK]
ID=<int> COLOR=<char> WIDTH=<int> HEIGHT=<int> INIT_X=<int> INIT_Y=<int> GEOMETRY=<bool[WIDTHxHEIGHT]>
...

[WALL]
....

[EXIT]
COLOR=<char> X=<int> Y=<int> ORIENTATION=<H,V> LI=<int> LF=<int> STEP=<int>

[GATE]
COLOR=<char> X=<int> Y=<int> ORIENTATION=<H,V> CI=<int> CF=<int> STEP=<int>
```

El archivo se divide en 5 secciones metadatos (META), descripción de bloques (block), descrición de paredes (WALL), descrición de salidas (EXIT), descripción de compuertas (GATE). E los bloques se describen sus propiedades incluso geometria. Esta última es un arreglo contiguo de valores booleanos que deben reordenarse en una matriz de $WIDTH \times HEIGHT$. pueden haber tantas filas como bloque hayan en el tablero. La sección WALL describe las murallas del tablero e incluye salidas y compuertas sin identificarlas. Esto se escribe como una matriz de $WIDTH \times HEIGHT$ con valores '' para pared y espacio para no pared. Las compuertas se describen en la sección GATE y describen cada compuerta que puede haber en una murralla.

Un ejemplo:

```
NAME = SIMPLE1
WIDTH = 8
HEIGHT = 8
STEP_LIMIT = 50

[BLOCK]
1 COLOR=a WIDTH=2 HEIGHT=2 INIT_X=4 INIT_Y=4 GEOMETRY=1 1 1 1
```

```
[WALL]
########
#      #
#      #
#      #
#      #
#      #
#      #
########

[EXIT]
COLOR=a X=2 Y=7 ORIENTATION=V LI=2 LF=2 STEP=1

[GATE]
```

En el ejemplo anterior se dispone un bloque de $2x2$ y una salida ubicada en la columna 7. La secuencia de solucion del problema es: R1,1 ; es decir en 1 paso toca la salida y sale. La visualización de esto es:

```
########
#      #
#      #
#      #
#   aa a
#   aa a
#      #
########

########
#      #
#      #
#      #
#    aaa
#    aaa
#      #
########

########
#      #
#      #
#      #
#      a
#      a
#      #
########
```

En este caso, el programa además entrega:

```
Tiempo resolución: 1[mseg]
Solución encontrada.
Pasos:
R1,1
```

El entregable debe ser un archivo comprimido de nombre ApellidoNombreRUT.zip

- Un makefile que compile programa principal y programas de test con compilación separada.
- Un test por cada clase generada. Debe comprobar que efectivamente la clase funcione.
- Cada clase son 2 archivos (.cpp) y (.h), que deben ser compilados en forma separada.

- Un main.cpp con un menu para seleccionar archivo de entrada y otras opciones descritas en este enunciado.
- pdf de informe
- varios ejemplos de entrada.

La carpeta comprimida cuyo nombre corresponda a ApellidoNombreRUT.zip (u otra compresión). Esta carpeta contiene archivos Header (.h), clases (.cpp), programas principales (main.cpp y test_XX.cpp), al menos un test por clase, un makefile con compilación separada. Toda clase debe ser implementada por separado en dos archivos (.h y .cpp). El nombre de una clase siempre comienza en mayúscula y el archivo que la implementa tiene su nombre. No se permite definir funciones fuera de las clases y cada clase debe servir a un solo propósito o abstracción.

# 5 Rúbrica

1. (10%) Informe : contiene una descripción de la estrategia de resolución que se utilizó y el porqué su implementación es eficiente para encontrar la solución.
2. (40%) Código : se revisa si compila (0 pts si no compila), si se implementan todas las funcionalidades descritas anteriormente, si logra ejecutarse en los ejemplos entregados, si tiene la estructura indicada (clases+test+main+makefile), si se efectúa compilación separada, si está todo descrito por clases y no hay funciones sueltas salvo la función main, si el makefile opera bien, si el código se entiende (comentarios, variables con nombres descriptivos, identación, sin repeticiones forzadas), si se encuentra 1 test adecuado por clase, si el programa se ejecuta sin problemas y efectúa las operaciones que se espera de la tarea (si no ejecuta en ningún caso 0 pts). **Si tiene funciones fuera de las clases salvo la función main tiene 0 puntos.** No puede utilizar contenedores de STL, las estructuras de datos deben ser implementadas.
3. (50%) Programa correcto y eficiencia (**0 pts si no funciona o no tiene ningún intento de hacerlo eficiente.**): Se revisa que no existan errores de lógica y/o ejecución, que entregue los resultados correctos en todos los casos (10%). Se revisa que se haya implementado el problema de manera eficiente (40% y estructuras de datos eficientes). Por eficientes se refiere a que operaciones de búsqueda, inserción, borrado, pop, push sean al menos $O(\log(N))$. Lo anterior no es suficiente ya que además deberá tener una componente heurística para acelerar la búsqueda de la solución (si no tiene esta parte se tiene 0 ptos en este ítem).
4. Los puntos anterior configuran la revisión de la entrega de tarea. La nota anterior se contrasta con una interrogación oral sobre el código entregado. En proporción 70% nota oral y 30% nota de entrega. En la interrogación se revisa si el alumno tiene pleno conocimiento de lo que entregó, si es así en su interrogación oral tendrá la misma nota que la corrección. Si no sabe como funciona el código se descuenta como si no lo hubiese entregado.
5. la nota final después de interrogación tiene 10% de nota por revisión de código de terceros.

# 6 Entrega

27 de Abril a las 11:55PM entrega que incluye lo anterior. 1 punto por día de atraso considerando entrega hora/fecha posterior a la indicada. Dentro entrega de la revisión de a pares. **El archivo entregable es una carpeta con ApellidoNombreRUT del alumno y comprimida, por ejemplo en un archivo zip como ApellidoNombreRUT.zip con los archivos requeridos.**

vía Classroom